When Recurrence Does Not Adapt: Probing for Implicit System Identification in Domain-Randomized Locomotion

Samuele Carrea
Politecnico di Torino
334035
s334035@studenti.polito.it

Alessandro Faletto
Politecnico di Torino
333479
s333479@studenti.polito.it

***Abstract*—Uniform Domain Randomization (UDR) trains a single policy without memory, so that it works over a range of different dynamics. However, such a policy cannot adapt inside an episode. A recurrent policy could in principle solve this problem, because it can infer the dynamics parameters from the past observations. This ability is often called implicit system identification. We test this idea on the MuJoCo Hopper sim-to-sim benchmark, where the source and the target domain differ only in the torso mass. We train a feedforward and a recurrent PPO agent with the same settings on six seeds, we train a supervised probe that tries to predict the true link masses from the LSTM hidden state, and we train an oracle policy that receives the true masses as input. All our results are negative, and they agree with each other. The feedforward policy is better than the recurrent one in every condition we tested; UDR does not close the domain gap for either architecture; the oracle does not gain anything from knowing the masses; and the mass information that we can decode from the hidden state is not reliably higher than what a randomly initialized LSTM already gives. The control with untrained recurrent weights is the key part of the analysis: without it, we would have reported an effect of the architecture as if it was learned identification. We also observe that decodability follows how well a policy converged, which is consistent with the idea that these parameters are simply not relevant for the task.**

***Index Terms*—sim-to-real transfer, domain randomization, recurrent policies, system identification, probing, negative results**

I. Introduction

Reinforcement learning for robotics is usually done in simulation, where interaction is cheap and safe. The policy is then deployed on real hardware, whose dynamics are never exactly the same as the simulator. This difference is called the sim-to-real gap [1]. Domain randomization is the standard way to reduce it: instead of training on a single simulator, the agent is trained on a distribution of simulators, so that the real system is covered as one sample among many [2], [3].

We study this problem in the sim-to-sim setting given by the course project. We use two variants of the MuJoCo [4] Hopper: a *source* domain, where training happens, and a *target* domain, which plays the role of the real world. The only difference between them is the torso mass, which is 1 kg heavier in the target. Uniform Domain Randomization (UDR) is applied to the other three link masses, because the assignment does not allow randomizing the torso.

UDR has a structural limitation, which the assignment itself points out: a policy without memory sees a single state and cannot know which member of the randomized family it is controlling at that moment. It can only learn one conservative behaviour that works on average over the whole distribution. A recurrent policy does not have this limitation. In principle it can collect evidence about the dynamics during the first steps of an episode and then adapt its behaviour to that estimate. This is the mechanism behind RL^2-style adaptation [5], the recurrent policies used for in-hand manipulation [6] and, in an explicit form, Rapid Motor Adaptation [7]. If this happens, we should see it in two ways: better transfer, and mass information that can be decoded from the hidden state.

We tested both, and neither of them holds. This paper reports a negative result and, more importantly, the controls that make this negative result interpretable and not only disappointing. Our contributions are:

1) A controlled comparison between feedforward and recurrent PPO with the same training budget, the same hyperparameters and the same seeds, over six seeds and four training conditions (Sec. V-A).
2) A supervised probe that decodes the randomized link masses from the LSTM hidden state, with a randomly initialized recurrent encoder as a control (Sec. V-E). The control changes the conclusion that the probe alone would support.
3) An oracle policy that receives the true masses in its observation. This gives an upper bound on how much any identification mechanism could be worth (Sec. V-C).
4) A cross-mass evaluation, to check whether recurrence helps when the parameter that varies is the one that UDR actually randomizes (Sec. V-D).

II. Related Work

Dynamics randomization. The idea of randomizing simulator parameters to obtain policies that transfer without fine-tuning on the real robot was introduced for visual inputs [2] and later extended to dynamics parameters such as masses,

TABLE I
LINK MASSES IN THE TWO DOMAINS (KG). UDR RANDOMIZES THE THREE MASSES THAT ARE IDENTICAL ACROSS DOMAINS; THE ONE THAT DIFFERS CANNOT BE RANDOMIZED.

Domain	torso	thigh	leg	foot
Source	2.665	4.058	2.781	5.316
Target	3.665	4.058	2.781	5.316
Randomized by UDR	no	yes	yes	yes

friction and damping [3]. The standard formulation maximizes the expected return over a distribution of dynamics. This gives a single policy that is robust over the whole support, instead of a policy specialized on one member of it.

Implicit and explicit adaptation. Peng et al. [3] report that a recurrent policy trained with dynamics randomization transfers better than a feedforward one, and they explain this with an online system identification done by the memory. Andrychowicz et al. [6] make the same argument for in-hand manipulation, and support it by decoding object properties from the LSTM state. RMA [7] makes the mechanism explicit: a separate module estimates a latent representation of the environment parameters, and the policy is conditioned on it. Yu et al. [8] instead identify the parameters online and then select a parameter-conditioned policy.

Probing. Training a small supervised model on frozen internal representations, in order to test what information they contain, was formalized for classifiers [9] and is now a common tool in representation analysis. The small capacity of the probe is part of the argument: if even a weak model succeeds, then the information was already available in an (almost) linear form.

Probing needs controls. Hewitt and Liang [10] show that the accuracy of a probe alone does not tell us much, because a probe with enough capacity can learn the task by itself. What matters is the difference between the representation we are interested in and a control representation with the same structure. We interpret our probe results in this way. This is also the reason for our main control: an untrained recurrent network is still a random non-linear projection of the input history, that is, a reservoir in the sense of echo state networks [11], and a lot can be decoded from it even if nothing was learned.

III. METHOD

A. Domain gap and what UDR actually randomizes

Table I reports the link masses of the two domains. The only difference is the torso. UDR is applied to thigh, leg and foot, and each mass is sampled at every episode reset from $\mathcal{U}\left((1-\alpha)\, m_i,\ (1+\alpha)\, m_i\right)$, with $\alpha = 0.15$ by default and m_i the nominal source value.

This observation is structural, and it constrains everything that follows. The parameters that vary during training are exactly the parameters that are *identical* in source and target, while the parameter that creates the domain gap is never seen varying. So any mechanism that infers the randomized parameters, even if it works perfectly, recovers information that by construction says nothing about the gap. We do not consider this a mistake in the experimental design, because the constraint comes from the assignment. We use it as a hypothesis about *why* the mechanisms we test fail, and we test this hypothesis directly with the oracle and with the cross-mass evaluation.

B. Recurrent policy

We replace the feedforward PPO policy [12] with RecurrentPPO, where the actor and the critic each have a separate LSTM layer of 128 units placed before the policy and value heads; we tested the shared-LSTM variant and found it performed worse. The hidden state is reset at every episode boundary, so any estimate of the dynamics has to be rebuilt inside the episode. Apart from the recurrent layer, the two architectures use the same network sizes, the same hyperparameters, the same seeds and the same training budget, so that any difference can be attributed to recurrence only.

C. Probe

To test whether the recurrent state encodes the dynamics, we freeze a trained UDR RecurrentPPO policy and we run it in the source environment with randomized masses. Every 5 steps we save the concatenation of hidden and cell state, $h_t \oplus c_t \in \mathbb{R}^{256}$, together with the true masses of that episode. Then we train a small MLP with two hidden layers of 32 units to regress the three masses from that vector. We keep the probe small on purpose, following [9]: it should read information that is already there, not extract it.

We report the performance as the percentage reduction of the mean absolute error with respect to a baseline that always predicts the mean of each mass in the training set. This is the correct null model: the masses are sampled from a fixed uniform distribution, so predicting its mean is what a probe with no access to any dynamical information can do.

D. Oracle

The probe tells us whether the parameters *can* be recovered. It does not tell us whether recovering them is useful. The oracle answers this second question. We train a feedforward PPO policy whose observation is augmented with the true masses, so that no inference is needed. We use two variants: `links`, which appends thigh, leg and foot, and `all`, which appends the torso as well. The first one is the upper bound of any mechanism that identifies the randomized parameters. The second one checks whether even the parameter responsible for the domain gap would help, if it was known.

E. Controls

We use four controls on the probe result.

Random encoder (reservoir). We collect the same trajectories with the same acting policy, but the hidden states are read from a separate LSTM with the same architecture, randomly initialized and never trained, which receives the same observation stream. Since the acting policy does not

change, the distribution of the trajectories is the same, and the only difference is whether the encoder was trained or not. This separates the contribution of learning from the contribution of recurrence as a random non-linear filter [10], [11]. We use three independent encoder initializations.

Timestep-only. A probe that receives only the timestep index t. This detects survivorship bias: if episodes with certain masses live longer, then t alone would already be predictive.

Shuffled labels. The mass targets are permuted across episodes. This detects leakage through the episode-wise split.

Zero-history. The probe performance restricted to $t = 0$, where the hidden state does not contain any dynamical evidence yet, so the performance has to go back to the baseline.

IV. Experimental Setup

All the policies are trained with Stable-Baselines3 [13] and its `sb3-contrib` extension, for 10^6 timesteps on 8 parallel environments, with learning rate 3×10^{-4}, $n_{\text{steps}} = 2048/n_{\text{envs}}$, batch size 64, 10 epochs per update, $\gamma = 0.99$, GAE $\lambda = 0.95$ and clipping range 0.2. Observations and rewards are normalized with running statistics. Episodes are limited to 500 steps, so the returns are not directly comparable with the standard Hopper benchmarks. Every configuration is trained with six seeds when the available compute allowed it, and with three seeds otherwise. All policies are evaluated deterministically over 50 episodes, on both source and target.

The probe dataset contains 450 episodes per policy seed, sampled every 5 steps. It is evaluated with 5-fold cross-validation split *by episode*, so that two samples of the same episode never appear in the training and in the validation fold at the same time. The probe is trained for 150 epochs with Adam, learning rate 10^{-3} and batch size 256. Every dataset stores which checkpoint and which normalization statistics it was generated from, so that we can verify that a trained dataset and its control are really paired.

V. Results

A. Recurrence does not improve transfer

Table II and Fig. 1 report the main comparison. We can make three observations.

First, the feedforward policy is better than the recurrent one in every matched condition, on both domains, and with a much smaller variance across seeds. With UDR the gap on the source domain is large (1706 against 1120), and it does not close on the target.

Second, UDR does not close the domain gap, with either architecture. For the feedforward policy the target return goes from 853 ± 523 without randomization to 891 ± 349 with it, so the difference is well inside the noise across seeds. Changing the randomization amplitude does not help either. With a single recurrent seed, $\alpha = 0.05$, 0.25 and 0.50 give target returns of 832, 913 and 1084. All of them are below the upper bound obtained by training on the target, and all of them are inside the variability that we observe across seeds at fixed α.

Third, the variance is itself a result. The target return of the same configuration goes from a few hundred to more than 1600 depending only on the seed. In this setup, a successful transfer looks more like an occasional event than like a property of the method, and a comparison based on a single seed would not be interpretable.

TABLE II
Mean return over 50 evaluation episodes, averaged across seeds (n = number of seeds; std across seeds, or across episodes when $n = 1$). FF denotes feedforward PPO, LSTM denotes RecurrentPPO.

Training condition	Policy	n	$\rightarrow$ source	$\rightarrow$ target
No rand., source	FF	6	1723 ± 53	853 ± 523
No rand., source	LSTM	3	1325 ± 171	1169 ± 358
UDR ($\alpha = 0.15$)	FF	6	1706 ± 73	891 ± 349
UDR ($\alpha = 0.15$)	LSTM	6	1120 ± 265	745 ± 342
Oracle, links	FF	3	1430 ± 256	1150 ± 290
Oracle, all masses	FF	1	895 ± 89	1185 ± 36
No rand., target (UB)	FF	3	1630 ± 40	1603 ± 106
No rand., target (UB)	LSTM	3	1293 ± 132	1306 ± 131

B. The gap is not only a matter of training budget

A reasonable objection is that the recurrent policy is simply under-trained. The training curves support this only in part, and in a way that does not save the hypothesis. The feedforward runs saturate around 1600–1700 within 300–400k steps and then stay there. The recurrent runs oscillate between about 1000 and 1450 for the whole million steps without stabilizing, and with much wider bands across seeds.

So recurrence does have an optimization cost, and this cost is not paid back inside this budget; with a longer budget the gap could become smaller. However, this explanation cannot cover the rest of the paper. It does not explain why the oracle, which is feedforward and converges normally, gains nothing from knowing the masses, and it does not explain why the recurrent policy loses against the feedforward one in the cross-mass evaluation of Sec. V-D, where the parameter that varies is the one that can be inferred. For this reason we report the optimization cost as a real but partial explanation, which is also confounded with the effect we wanted to measure.

C. Perfect knowledge of the randomized masses is not useful

The oracle isolates the value of the information, without the difficulty of inferring it. A feedforward policy that receives the true thigh, leg and foot masses in its observation reaches 1430 ± 256 on source and 1150 ± 290 on target. It does not beat the plain feedforward baseline on source (1723), and its advantage on target is well inside the seed variance of every other configuration.

We consider this our strongest single result, because here a mechanism is not failing: the mechanism is skipped completely. The information about the randomized parameters is given to the policy for free, and it does not buy anything. Whatever a perfect implicit identifier could obtain is bounded from above by this number.

The `all` variant, which also reveals the torso mass, is consistent with this: 1185 on target with a single seed, which

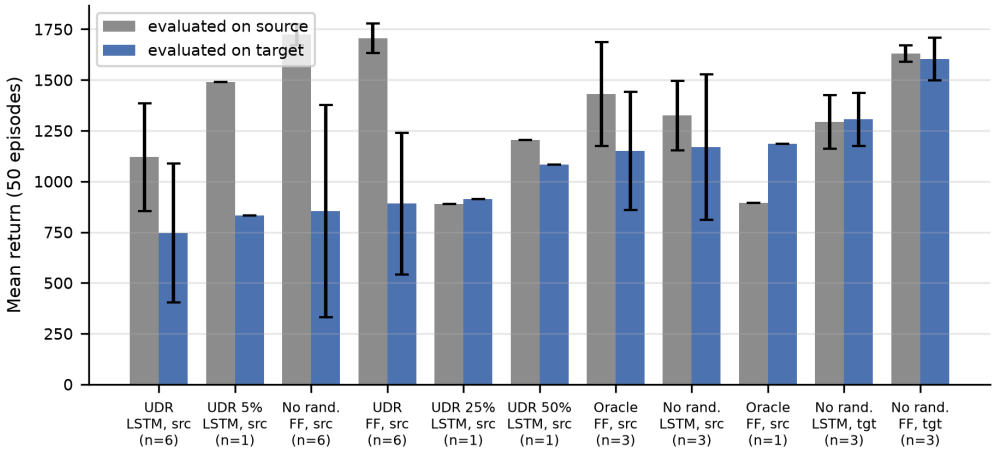

Fig. 1. Mean return over 50 evaluation episodes on the source (grey) and target (blue) domains, for every trained configuration, sorted by target return. Error bars are standard deviations across seeds where more than one seed is available, and across evaluation episodes otherwise. Feedforward policies (FF) are better than their recurrent counterparts (LSTM) in every matched comparison.

is not better than the `links` oracle. Knowing the parameter that actually differs does not help either, because the policy is still trained only on source torso values and never experiences the target one. In other words, knowing a parameter is not the same as having trained on it.

D. Recurrence does not help even on the randomized parameters

If the only problem was that the torso is never randomized, then recurrence should still help when the parameter that varies is one of those covered by UDR. We tested this by evaluating the trained checkpoints on environments where thigh, leg and foot are scaled together by a factor from 0.70 to 1.30, keeping the torso at its source value (Fig. 2, left).

The feedforward policy wins at every scale, and the margin becomes larger outside the training range: at scale 1.30 it reaches 1283 ± 398 against 1032 ± 247. The recurrent policy is almost flat over the whole sweep, which is what we expect from a policy that learned a single behaviour and does not condition it on the dynamics.

Scaling one link at a time (Fig. 2, right; feedforward, seed 42) explains why. On this seed, thigh and leg are essentially flat over the whole range (variation $< 2\%$), while only the foot mass shows monotone degradation ($1827 \rightarrow 1644$). This per-link analysis is indicative; the pattern is consistent across the 6-seed ensemble when masses are scaled jointly (Fig. 2, left). So with perturbations of $\pm 15\%$, two of the three randomized masses have almost no effect on the task. There is not much that identification could be useful for, both when it is done implicitly and when the parameters are given for free.

E. The probe, and what the control does to it

Table III reports the probe results. If we read the first three rows alone, they look like a positive finding: the mass information can be decoded from the hidden state clearly above the mean-predictor baseline, more strongly for the foot and weakly for the leg, with the same ranking that the cross-mass evaluation gave independently. The basic controls hold on every dataset: a probe that uses only the timestep stays within $\pm 0.2\%$ of the baseline for seed 42, shuffled labels score below zero, and at $t = 0$ the performance goes back to the baseline. So the signal is not survivorship bias, it is not leakage, and it cannot be read before some dynamical evidence has accumulated.

The control changes this reading. An LSTM whose weights were never trained still projects the observation history in a non-linear way into 256 dimensions, and the probe reads that projection almost as well (Fig. 3): 17.8 ± 0.5, 5.0 ± 0.2 and 19.9 ± 0.8 for thigh, leg and foot, against 19.6 ± 12.3, 4.6 ± 2.6 and 24.9 ± 17.4 for the trained encoders. The trained means are not below the random ones, so we do not claim that training destroys the information. Our claim is weaker and better supported: *the trained representation does not give a reliable advantage over an untrained one with the same architecture*. This is exactly the comparison that Hewitt and Liang [10] consider the only informative one. The behaviour over time does not help the learning hypothesis either: the shape of the decodability over time, including the early peak on the foot and its decay afterwards, is reproduced by the random encoder as well as by the trained ones.

If we average over the trained seeds, we hide a more interesting fact. The three random encoders agree with each other within one percentage point, while the three trained seeds differ by up to 24 points. So what changes on the trained side is not the encoder initialization. It is the policy.

These three seeds are very different in how well they solved the task, with target returns of 395, 821 and 1211, and the decodability follows the same ordering in the opposite direction on all three masses, fitted independently (Fig. 4): Pearson r of -0.982, -0.824 and -0.984. With $n = 3$ no single correlation is significant ($p > 0.1$ in all the cases) and we do not claim that it is. The evidence we use is that the

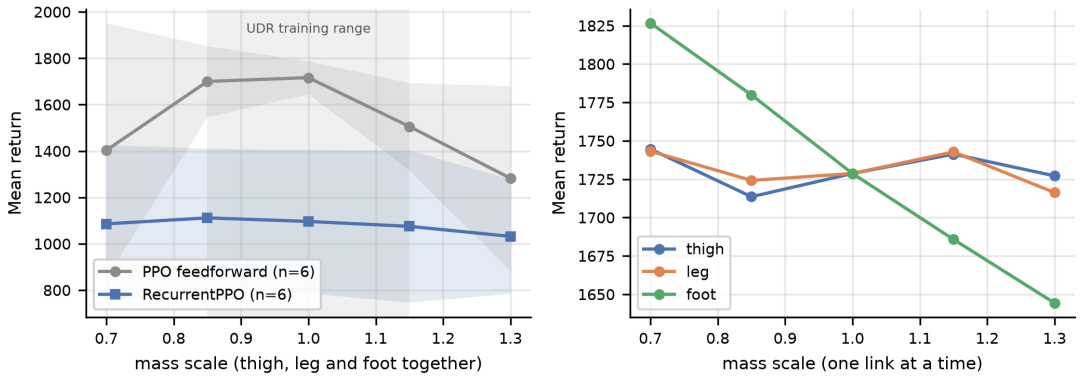

Fig. 2. Cross-mass evaluation. Left: return as a function of a common scaling of thigh, leg and foot, mean ± std over six seeds per architecture; the shaded band marks the UDR training range. The feedforward policy is better at every scale, including outside the training range. Right: scaling one link at a time (feedforward, seed 42): thigh and leg are flat, only the foot mass changes the return in a measurable way.

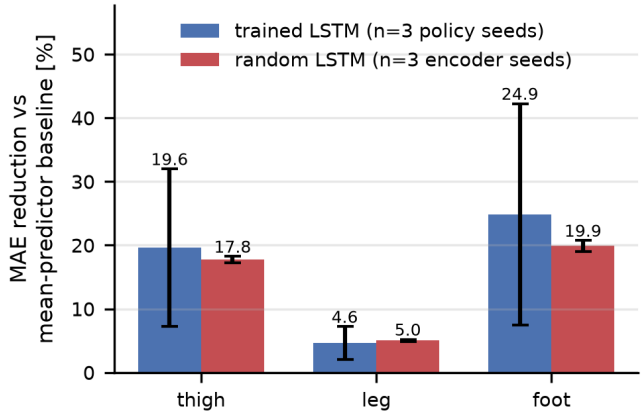

Fig. 3. Probe performance on the trained LSTM (three policy seeds) against an untrained, randomly initialized LSTM with the same architecture, fed with the same observation stream (three encoder seeds). Bars are means, error bars are standard deviations across runs. The trained representation does not give a reliable advantage, and its spread is one order of magnitude larger.

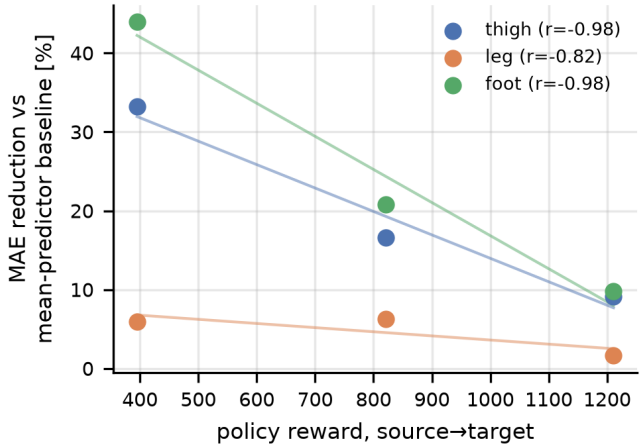

Fig. 4. Decodability against policy return for the three UDR RecurrentPPO seeds, per mass, with least-squares fits. The sign is negative on all three masses, fitted independently. With $n = 3$ no single correlation is significant; our evidence is that the sign is the same on all the masses.

TABLE III

PROBE PERFORMANCE: PERCENTAGE REDUCTION IN MEAN ABSOLUTE ERROR RELATIVE TO A MEAN-PREDICTOR BASELINE, 5-FOLD CROSS-VALIDATION SPLIT BY EPISODE. "RETURN" IS THE POLICY'S SOURCE→TARGET RETURN; SEEDS ARE ORDERED BY HOW WELL THEY CONVERGED. THE LAST ROW IS THE PEARSON CORRELATION BETWEEN RETURN AND DECODABILITY ACROSS THE THREE SEEDS.

Encoder	return	thigh	leg	foot
Trained, seed 7	395	33.2	6.0	43.9
Trained, seed 123	821	16.6	6.2	20.8
Trained, seed 42	1211	9.1	1.6	9.8
Random, $n = 3$	—	17.8 ± 0.5	5.0 ± 0.2	19.9 ± 0.8
Pearson r	—	-0.982	-0.824	-0.984

sign is the same on three targets fitted separately, and that it points in the same direction as the oracle result.

The interpretation that we consider best supported is that what removes the mass information from the state is convergence, and not training in itself. The seed that converged best is the only one that stays clearly below the reservoir baseline on every mass. The seed that barely converged still looks like an under-trained network, which has not compressed its state towards what the task needs yet, and it decodes better than a random one. A policy that has solved the task has no reason to keep information that it does not use, and the oracle tells us directly that this information is not useful.

VI. DISCUSSION

Our four experiments fail for different reasons, but we think that a single explanation covers all of them: *the parameters that UDR randomizes are almost irrelevant for this task, and none of them is the parameter that creates the domain gap.*

The oracle shows this directly. The cross-mass evaluation shows it structurally: within $\pm 15\%$, two of the three randomized masses do not change the optimal behaviour in a measurable way, and a single robust gait covers the whole

range, so a specialized behaviour would have nothing to gain. The probe shows it indirectly, through the correlation between convergence and the absence of decodable mass information. Finally, the comparison between the policies shows the cost of recurrence: it adds an optimization burden that should be paid back by adaptation benefits which, in this setup, do not exist.

We think that the methodological point is as important as the empirical one. Our probe, if we run it without the random-encoder control, gives exactly the numbers that one would report as evidence of implicit system identification: decodability well above the baseline, at chance level at $t = 0$, growing during the first steps of the episode, and higher for the mass that matters most for control. All of these properties are reproduced by an LSTM whose weights were never trained. So the property that looked like learning is a property of recurrence as a random projection of the input history. Here the difference between what a representation *contains* and what the network *uses* [10] is not a detail: it is the difference between the conclusion we would have reported and the conclusion that the evidence supports.

This does not contradict the literature that motivated our study. Peng et al. [3] and Andrychowicz et al. [6] randomize parameters that really change the optimal behaviour, over wider ranges and with much larger training budgets, and in the case of [7] with an architecture that is explicitly supervised to extract the latent parameters. Our result identifies conditions in which implicit adaptation does not appear, which is a complementary statement and not a contradictory one.

VII. Limitations

The correlation between return and decodability is based on three policy seeds. The fact that the sign is the same on three masses fitted independently is an indication, but $n = 3$ does not support a claim of significance, and we do not make one. More UDR RecurrentPPO seeds would be needed to test it properly, and the compute budget available for the project did not allow this.

The recurrent policy has not converged at 10^6 steps, so our comparison measures recurrence *at a fixed budget*, and not recurrence at convergence. A longer schedule, or a tuning specific for the recurrent architecture, could change the comparison between the policies, although, as we discussed above, it would not explain by itself the oracle and the cross-mass results.

The normalization statistics are saved only at the end of training, so they match the final checkpoint and not the intermediate best-return ones. We tried to re-evaluate the best-return checkpoints as a robustness check, and the results were not usable exactly for this reason: some intermediate checkpoints collapse to a return close to zero when they are evaluated with statistics that do not match. All the numbers in this paper come from final checkpoints. Fixing this would require saving the statistics at every improvement and training everything again, and we leave it as future work.

Finally, the variants on the randomization amplitude and the all-masses oracle use a single seed, so they are only indicative; the random encoder control uses three initializations, which we consider enough given the very small spread that we observe, but it is not an exhaustive estimate; and the episodes are limited to 500 steps, so the absolute returns cannot be compared with published Hopper results.

VIII. Conclusion

We asked whether a recurrent policy trained with uniform domain randomization performs implicit system identification, and whether this improves the sim-to-real transfer on the Hopper benchmark. The answer is negative in both cases, and the two negative answers are consistent with each other: the randomized parameters have no value for the task, as the oracle shows directly and the cross-mass evaluation explains structurally, so there is nothing that the memory could identify that would be worth identifying.

The methodological conclusion is that a probe without a matched untrained control cannot distinguish a learned representation from a by-product of the architecture. In our case the two are indistinguishable on every diagnostic that we measured, except the control itself.

For implicit adaptation to appear in this setting, the randomization should cover the parameter that actually differs between the domains, or it should use amplitudes wide enough that a single gait cannot cover the whole range. Both options are outside the constraints of this assignment, and in our opinion both are the natural next experiments.

References

[1] S. Höfer et al., "Perspectives on sim2real transfer for robotics: A summary of the R:SS 2020 workshop," *arXiv preprint arXiv:2012.03806*, 2020.

[2] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IROS*, 2017.

[3] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *ICRA*, 2018.

[4] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IROS*, 2012.

[5] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel, "RL^2: Fast reinforcement learning via slow reinforcement learning," *arXiv preprint arXiv:1611.02779*, 2016.

[6] M. Andrychowicz et al., "Learning dexterous in-hand manipulation," *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.

[7] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid motor adaptation for legged robots," in *Robotics: Science and Systems*, 2021.

[8] W. Yu, J. Tan, C. K. Liu, and G. Turk, "Preparing for the unknown: Learning a universal policy with online system identification," in *Robotics: Science and Systems*, 2017.

[9] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes," *arXiv preprint arXiv:1610.01644*, 2016.

[10] J. Hewitt and P. Liang, "Designing and interpreting probes with control tasks," in *EMNLP*, 2019.

[11] H. Jaeger, "The echo state approach to analysing and training recurrent neural networks," German National Research Center for Information Technology, Tech. Rep. 148, 2001.

[12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[13] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.